

Universidad Nacional Mayor de San Marcos
Facultad de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Sistemas



TRABAJO GRUPAL:

Gestión de Flujo de Aprobaciones - CAMPUS360

DOCENTE:

Andrade Mogollon, Teodoro Manuel

ASIGNATURA:

DISEÑO DE SISTEMAS DE INFORMACIÓN

SECCIÓN:

INTEGRANTES:

Condori Pari, Arnol
Huaranca Quilca, Gustavo Enrique
Moya Laureano, Jussel Enrique
Ninamango Mucha, Demian Joseph
Quispe Paricahua, Dante Jussie
Ticsihua De La Cruz, Erick Enerson

Lima - Perú
2026

ÍNDICE

1. Introducción a la Calidad del Código	3
1.1. Objetivos de la refactorización en el Módulo de Aprobaciones	3
1.2. Principios de diseño aplicados	3
2. Matriz General de Code Smells Identificados.....	4
2.1. Tabla resumen de hallazgos	5
3. Evidencia de Refactorización: Análisis Comparativo (Antes vs. Después)	6
3.1. Caso 1: Long Method y Código Espagueti (Complejidad Ciclomática)	6
3.1.1. Diagnóstico e Impacto	6
3.1.2. Código Original (Antes)	6
3.1.3. Técnica de Refactorización Aplicada	7
3.1.4. Código Refactorizado (Después)	7
3.2. Caso 2: Fat Controller (Violación del Principio de Responsabilidad Única - SRP).....	8
3.2.1. Diagnóstico e Impacto	8
3.2.2. Código Original (Antes)	8
3.2.3. Técnica de Refactorización Aplicada	9
3.2.4. Código Refactorizado (Después)	9
3.3. Caso 3: Duplicated Code (Violación del Principio DRY)	10
3.3.1. Diagnóstico e Impacto	10
3.3.2. Código Original (Antes)	10
3.3.3. Técnica de Refactorización Aplicada	11
3.3.4. Código Refactorizado (Después)	12
3.4. Caso 4: Too Many Parameters y Obsesión Primitiva	12
3.4.1. Diagnóstico e Impacto	12
3.4.2. Código Original (Antes)	13
3.4.3. Técnica de Refactorización Aplicada	13
3.4.4. Código Refactorizado (Después)	14
3.5. Caso 5: Message Chains (Violación de la Ley de Demeter)	15
3.5.1. Diagnóstico e Impacto	15
3.5.2. Código Original (Antes)	15
3.5.3. Técnica de Refactorización Aplicada	16
3.5.4. Código Refactorizado (Después)	16
4. Verificación y Métricas de Calidad	17
4.1. Validación de la refactorización (Pruebas Unitarias)	17
4.2. Mejoras medibles en la arquitectura	17

1. Introducción a la Calidad del Código

El desarrollo de software no concluye con la entrega de un sistema funcional; requiere una evaluación constante de su estructura interna para asegurar su viabilidad a largo plazo. En el contexto del sistema Campus360, el presente documento detalla el proceso sistemático de mejora continua aplicado al código fuente, garantizando que el diseño interno sea tan robusto como su comportamiento externo.

1.1. Objetivos de la refactorización en el Módulo de Aprobaciones

Durante las fases iniciales de desarrollo del Módulo de Aprobaciones, la prioridad fue alcanzar un Producto Mínimo Viable (MVP) funcional. Sin embargo, este enfoque generó deuda técnica en forma de Code Smells (malos olores en el código), tales como métodos extensos, alta complejidad ciclomática y acoplamiento indebido.

El objetivo principal de esta fase de refactorización fue reestructurar el código fuente sin alterar su comportamiento externo, buscando una mejora medible en tres aspectos clave: la reducción de la complejidad, la eliminación de la duplicación y el aumento de la legibilidad. Específicamente, este proceso se centró en la identificación y corrección de al menos 5 smells críticos, preparando el módulo para integraciones futuras, facilitando la creación de pruebas automatizadas y asegurando su mantenibilidad en el tiempo.

1.2. Principios de diseño aplicados

Para guiar el proceso de refactorización, el sistema se estructuró siguiendo una arquitectura limpia definida por las siguientes capas: Capa de Presentación (API REST), Capa de Aplicación/Servicios, Capa de Dominio y Capa de Infraestructura/Persistencia. Sobre esta base, se aplicaron de manera estricta los siguientes principios universales de diseño:

- Principios SOLID: Se implementaron rigurosamente para lograr alta cohesión y bajo acoplamiento.
 - Principio de Responsabilidad Única (SRP): Se erradicaron los controladores sobrecargados (Fat Controllers), delegando la orquestación a la capa de aplicación y las validaciones a esquemas de dominio.
 - Principio Abierto/Cerrado (OCP) y de Inversión de Dependencias (DIP): Se aplicaron mediante el uso de interfaces y abstracciones (como el patrón Strategy).
- Principio DRY (Don't Repeat Yourself): Se identificaron y centralizaron bloques de código duplicado, consolidándolos en componentes reutilizables (DTOs).
- Ley de Demeter (Principio del menor conocimiento): Se corrigieron vulnerabilidades de diseño relacionadas con el acoplamiento estructural, como las cadenas de mensajes largas (Message Chains).

2. Matriz General de Code Smells Identificados

En alineación con los estándares de calidad del proyecto, se identificaron 5 *Code Smells* principales en el código base inicial. A continuación, se detalla su ubicación, impacto y la técnica de refactorización exacta que se aplicó para mitigarlos:

2.1. Tabla resumen de hallazgos

Tipo de <i>Code Smell</i>	Archivo / Línea (Aprox.)	Impacto Técnico en el Sistema	Refactoring Aplicado
1. Long Method	services/aprobaciones_service.py	Alta complejidad ciclomática al calcular los tiempos límite (SLA) sumando días a través de múltiples sentencias if/elif/else. Violaba el Principio Abierto/Cerrado (OCP).	(Patrón Strategy): Se creó la interfaz SlaStrategy y sus clases concretas (TramiteRegularStrategy, etc.) para delegar el cálculo.
2. God Object (Fat Controller)	api/approval_controller.py	El controlador asumía múltiples responsabilidades (acceso al ORM, mapeo, reglas de SLA), imposibilitando las pruebas aisladas.	(Patrón Facade): Se extrajo la lógica de orquestación a la clase BandejaAprobacionFacade, dejando al controlador únicamente con el enrutamiento HTTP.
3. Duplicated Code	api/approval_controller.py	Existían bloques de código manual idénticos encargados de transformar las entidades ORM (SQLAlchemy) en diccionarios JSON para las respuestas de distintos endpoints.	Extract Method / Introduce DTO: Se delegó la transformación a esquemas de Pydantic (SolicitudDTO), centralizando el mapeo de salida en un solo lugar.
4. Too Many Parameters	repositories/solicitud_repository.py	Los métodos para registrar dictámenes o derivaciones recibían múltiples variables primitivas sueltas (strings, enteros, booleanos), dificultando la lectura y las validaciones.	Introduce Parameter Object: Se agruparon los parámetros en objetos de transferencia validados con Pydantic (DictamenInput y DerivacionInput).
5. Message Chains	api/approval_controller.py	El controlador accedía a los datos encadenando relaciones del ORM (ej. solicitud.estado_actual.tipoEstado), acoplándose fuertemente a la estructura interna de la base de datos.	Hide Delegate: La Fachada y el Repositorio ahora aplanan y mapean estos datos, entregando al controlador un objeto plano donde ya no necesita "navegar" por las relaciones.

3. Evidencia de Refactorización: Análisis Comparativo (Antes vs. Después)

3.1. Caso 1: *Long Method* y Código Espagueti (Complejidad Ciclomática)

3.1.1. Diagnóstico e Impacto

En la fase inicial, el cálculo de los días límite (Acuerdo de Nivel de Servicio - SLA) y la prioridad operativa se realizaba mediante una extensa cadena de condicionales if/elif/else que evaluaban el nombre del trámite directamente en el servicio principal.

- **Impacto:** Alta complejidad ciclomática (Spaghetti Code).
- **Violación:** Principio Abierto/Cerrado (OCP). Cada vez que se creaba un nuevo tipo de trámite, se debía modificar el código core, arriesgando la estabilidad de los flujos ya existentes.

3.1.2. Código Original (Antes)

Fragmento que evidencia la alta complejidad ciclomática y el acoplamiento:

```
# app/services/aprobaciones_service.py (Versión Inicial con
Code Smell)
from datetime import datetime, timedelta

def crear_solicitud_estudiante(tipo_tramite: str,
solicitante: str):
    # SMELL: Long Method y múltiples condicionales anidados
    if tipo_tramite == "Constancia de Estudios" or
tipo_tramite == "Historial Académico":
        horas_sla = 72
        prioridad = "NORMAL"
    elif tipo_tramite == "Subsanación de Notas" or "Urgente"
in tipo_tramite:
        horas_sla = 24
        prioridad = "ALTA"
    elif tipo_tramite == "Rectificación de Matrícula":
        horas_sla = 48
        prioridad = "NORMAL"
    else:
        # Caso por defecto
        horas_sla = 120
        prioridad = "BAJA"

    fecha_limite = datetime.now() +
timedelta(hours=horas_sla)
    # Continúa lógica de instanciación...
```

3.1.3. Técnica de Refactorización Aplicada

Se aplicó la técnica Replace Conditional with Polymorphism. Se extrajo cada rama de los condicionales hacia clases independientes que comparten una interfaz común (Patrón Strategy), y se delegó la selección a un inyector de dependencias estático (**SlaStrategyResolver**).

3.1.4. Código Refactorizado (Después)

El servicio principal se limpió por completo. Ahora, la lógica vive aislada y es fácilmente extensible:

```
# app/services/sla_strategy.py (Versión Refactorizada)
"""
Capa de Servicios: Lógica de Comportamiento (Strategy
Pattern).
Aplica el Patrón Strategy para el cálculo dinámico de
Acuerdos de Nivel
de Servicio (SLA) y prioridades. Garantiza el principio
Abierto/Cerrado (OCP).
"""
from datetime import datetime, timedelta
from abc import ABC, abstractmethod

class SlaStrategy(ABC):
    """
    Contrato base (Interfaz). Define la familia de algoritmos
    intercambiables para el cálculo de SLAs operativos.
    """
    @abstractmethod
    def calcular_sla(self) -> datetime:
        pass

    @abstractmethod
    def obtener_prioridad(self) -> str:
        pass

class TramiteRegularStrategy(SlaStrategy):
    """Estrategia Concreta: Trámites estándar con SLA de 72
    horas (Prioridad NORMAL)."""
    def calcular_sla(self) -> datetime:
        return datetime.now() + timedelta(hours=72)

    def obtener_prioridad(self) -> str:
        return "NORMAL"

class TramiteUrgenteStrategy(SlaStrategy):
    """Estrategia Concreta: Trámites críticos con SLA de 24
    horas (Prioridad ALTA)."""
    def calcular_sla(self) -> datetime:
```

```

        return datetime.now() + timedelta(hours=24)

    def obtener_prioridad(self) -> str:
        return "ALTA"

class SlaStrategyResolver:
    """
    Inyector de Dependencias Estático. Desacopla la selección
    de la estrategia
    del Factory, asegurando que la infraestructura cumpla con
    OCP y DIP.
    """
    @staticmethod
    def resolver(tipo_tramite: str) -> SlaStrategy:
        if "Extemporánea" in tipo_tramite or "Urgente" in
tipo_tramite:
            return TramiteUrgenteStrategy()
            return TramiteRegularStrategy()

```

3.2. Caso 2: *Fat Controller* (Violación del Principio de Responsabilidad Única - SRP)

3.2.1. Diagnóstico e Impacto

En la primera iteración de la API REST del Módulo de Aprobaciones, el endpoint encargado de listar la bandeja de solicitudes pendientes (/approvals/pending) asumía un exceso de responsabilidades. Además de recibir la petición HTTP, el controlador consultaba al ORM para traer los datos, aplicaba la lógica de negocio para calcular el semáforo visual de SLA, y ejecutaba un algoritmo de ordenamiento multicriterio.

Diagnóstico: Este anti-patrón es conocido como Fat Controller (o God Object a nivel de clase). Presentaba muy baja cohesión, ya que mezclaba la capa de presentación con el núcleo del dominio.

Impacto: Violación del Principio de Responsabilidad Única (SRP). Cualquier cambio en la forma de ordenar la bandeja o en el cálculo del semáforo exigía modificar directamente el controlador de la API. Además, dificultaba aislar la lógica para crear pruebas unitarias sin una conexión activa a MySQL.

3.2.2. Código Original (Antes)

A continuación, se reconstruye el estado inicial del *endpoint*, evidenciando la sobrecarga de lógica de negocio en la capa de presentación:

```

# app/api/approval_controller.py (Versión Inicial - "God
Object")
@router.get("/approvals/pending")
def listar_pendientes(db: Session = Depends(get_db)) :

```



```

# SMELL 1: Acceso directo a reglas de persistencia
solicitudes =
db.query(Solicitud).filter(Solicitud.estado_id == 1).all()

dto_list = []
# SMELL 2: Lógica de negocio (SLA) en el controlador
for sol in solicitudes:
    semaforo = "ROJO" if sol.slaObjetivo < datetime.now()
else "VERDE"
    dto = SolicitudDTO(
        id=sol.idSolicitud, alumno=sol.solicitante,
        tipo_tramite=sol.tipoSolicitud,
prioridad=sol.prioridad,
        semaforo_sla=semaforo
    )
    dto_list.append(dto)

# SMELL 3: Lógica de ordenamiento anidada en la
presentación
priority_order = {"ALTA": 1, "NORMAL": 2, "BAJA": 3}
dto_list.sort(key=lambda x:
(priority_order.get(x.prioridad, 99), x.semaforo_sla ==
"VERDE"))

return dto_list

```

3.2.3. Técnica de Refactorización Aplicada

Para sanear la API, se utilizó la técnica de refactorización Extract Class apoyada en el patrón Facade. Toda la lógica de obtención de datos crudos, cálculo de semáforos en tiempo real y aplicación de reglas de ordenamiento estricto fue extraída a **BandejaAprobacionFacade**. El controlador fue refactorizado para actuar únicamente como un orquestador ligero de tráfico HTTP.

3.2.4. Código Refactorizado (Después)

El archivo del controlador se redujo a su mínima expresión. Ahora cumple con el principio SRP, documenta su Caso de Uso explícitamente y delega la responsabilidad estructural:

```

# app/api/approval_controller.py (Versión Refactorizada
actual)
"""
Capa de Presentación: Controladores API (API Layer)
Orquesta las peticiones HTTP y mapea los Casos de Uso (CU)
del sistema,
delegando la lógica de negocio a la Capa de Servicios para
cumplir con SOLID (SRP).
"""

```

```

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from typing import List
from app.domain.schemas import SolicitudDTO
from app.config.database import get_db
from app.services.bandeja_facade import
BandejaAprobacionFacade

router = APIRouter()

@router.get("/approvals/pending",
response_model=List[SolicitudDTO])
def listar_pendientes(db: Session = Depends(get_db)):
    """
    CU-01: Listar Bandeja de Pendientes.
    Patrón Estructural: Utiliza BandejaAprobacionFacade para
encapsular la lógica
    compleja de ordenamiento por SLA y cálculo del semáforo.
    """
    fachada = BandejaAprobacionFacade(db)
    return fachada.obtener_bandeja_ordenada()

```

3.3. Caso 3: *Duplicated Code* (Violación del Principio DRY)

3.3.1. *Diagnóstico e Impacto*

Durante la auditoría del código de la capa de presentación (**app/api/approval_controller.py**), se detectó que múltiples endpoints (como listar pendientes, consultar historial y ver detalle) compartían bloques de código iterativo idénticos. La responsabilidad de estos bloques era transformar manualmente las entidades crudas del ORM (SQLAlchemy) en diccionarios JSON para ser enviados como respuesta HTTP al cliente frontend.

Diagnóstico: Este es el clásico Code Smell de Duplicated Code (Código Duplicado). La lógica de transformación y mapeo de datos estaba dispersa y repetida a lo largo de todo el controlador.

Impacto: Violación directa del principio DRY (Don't Repeat Yourself). Si la universidad solicitaba agregar o renombrar un campo en la respuesta de las solicitudes (por ejemplo, cambiar "alumno" por "solicitante"), el equipo de desarrollo tendría que buscar y modificar manualmente cada diccionario en cada uno de los endpoints, aumentando drásticamente el costo de mantenimiento y el riesgo de introducir errores humanos.

3.3.2. *Código Original (Antes)*

A continuación, se reconstruye cómo se veía la duplicación en el controlador antes de aplicar la refactorización. Se armaban diccionarios a mano repetidas veces:

```

# app/api/approval_controller.py (Versión Inicial con Code
Smell)

```

```

@router.get("/approvals/pending")
def listar_pendientes(db: Session = Depends(get_db)):
    solicitudes =
db.query(Solicitud).filter(Solicitud.estado_id == 1).all()
    resultado = []
    for sol in solicitudes:
        # SMELL: Mapeo manual repetitivo a diccionario
        resultado.append({
            "id": sol.idSolicitud,
            "alumno": sol.solicitante,
            "tipo_tramite": sol.tipoSolicitud,
            "prioridad": sol.prioridad
        })
    return resultado

@router.get("/approvals/history")
def consultar_historial(db: Session = Depends(get_db)):
    historicas =
db.query(Solicitud).filter(Solicitud.estado_id != 1).all()
    resultado = []
    for sol in historicas:
        # SMELL: El mismo mapeo manual duplicado en otro
endpoint
        resultado.append({
            "id": sol.idSolicitud,
            "alumno": sol.solicitante,
            "tipo_tramite": sol.tipoSolicitud,
            "prioridad": sol.prioridad
        })
    return resultado

```

3.3.3. Técnica de Refactorización Aplicada

Para eliminar esta redundancia, se aplicó la técnica Introduce DTO (Data Transfer Object). Se delegó la responsabilidad de la estructura de salida a esquemas fuertemente tipados utilizando la librería **Pydantic**.

Se creó el esquema **SolicitudDTO** en la capa de dominio. Esta clase centraliza el "contrato" de la API, ocultando la complejidad de la base de datos y exponiendo solo los datos procesados. Al habilitar **from_attributes=True**, el framework serializa automáticamente las entidades ORM, eliminando por completo la necesidad de armar diccionarios a mano.

3.3.4. Código Refactorizado (Después)

El mapeo iterativo fue erradicado de los controladores y centralizado en un único DTO validado:

```
# app/domain/schemas.py (Código Refactorizado)
"""
Capa de Dominio: Esquemas de Validación y DTOs (Data Transfer
Objects).
Desacopla los modelos de persistencia (ORM) de la interfaz de
red (JSON).
Aplica validaciones tempranas usando Pydantic para garantizar
las Reglas de Negocio.
"""
from pydantic import BaseModel, ConfigDict

# --- RESPUESTAS (OUTPUTS) ---
class SolicitudDTO(BaseModel):
    """
    DTO de Salida: Estructura pública enviada al cliente
    (Frontend).
    Oculta la complejidad de la BD y expone solo datos
    procesados (ej. semaforo_sla).
    """
    id: int
    alumno: str
    tipo_tramite: str
    estado: str
    prioridad: str
    semaforo_sla: str

    model_config = ConfigDict(from_attributes=True)
```

3.4. Caso 4: *Too Many Parameters* y Obsesión Primitiva

3.4.1. Diagnóstico e Impacto

Al analizar las firmas de los métodos responsables de las transiciones de estado — específicamente en la evaluación de secretaría y el registro de dictámenes por parte de los aprobadores— se observó que los controladores (`approval_controller.py`) y repositorios (`solicitud_repository.py`) recibían una lista extensa de variables primitivas sueltas (**id: int**, **decision: str**, **comentario: str**, **checklist_valido: bool**).

Diagnóstico: Esta estructura manifiesta dos Code Smells concurrentes: Too Many Parameters (Demasiados Parámetros) y Primitive Obsession (Obsesión Primitiva). El código confiaba ciegamente en tipos de datos básicos (strings y booleanos) que no tienen semántica de negocio por sí mismos.

Impacto: El sistema era vulnerable a estados inconsistentes y violaba el principio Fail-Fast (Falla Rápido). Al no haber un objeto que encapsulara y validara las reglas de negocio (como la RN-03, que exige un comentario obligatorio en caso de rechazo), los parámetros crudos viajaban por todas las capas, obligando a cada función a repetir validaciones manuales con if/else, ensuciando la capa de persistencia y dificultando la lectura del código.

3.4.2. Código Original (Antes)

A continuación, se reconstruye cómo lucía la firma de las funciones y la validación manual antes de la refactorización, evidenciando el exceso de parámetros y la lógica dispersa:

```
# app/repositories/solicitud_repository.py (Versión Inicial
con Code Smell)

# SMELL: Lista larga de parámetros primitivos sueltos
def derivar_solicitud(db: Session, solicitud_id: int,
area_destino: str, checklist_valido: bool, comentario: str =
""):

    # SMELL: Validación manual y repetitiva de reglas de
negocio en la capa incorrecta
    if checklist_valido is False:
        if not comentario or len(comentario.strip()) < 5:
            raise Exception("El comentario es obligatorio al
observar.")

    # Lógica de actualización ORM...
    pass

def registrar_dictamen(db: Session, solicitud_id: int,
decision: str, comentario: str = ""):
    if decision in ['OBSERVADO', 'RECHAZADO']:
        if not comentario or len(comentario.strip()) < 5:
            raise Exception("El comentario es obligatorio y
debe explicar el motivo.")

    # Lógica de actualización ORM...
    pass
```

3.4.3. Técnica de Refactorización Aplicada

Para blindar el sistema, se aplicó la técnica de refactorización Introduce Parameter Object (Introducir Objeto de Parámetro).

En lugar de pasar datos sueltos, se agruparon los parámetros cohesivos en Objetos de Transferencia de Datos (DTOs) de entrada validados mediante Pydantic: **DictamenInput** y **DerivacionInput**. Se trasladaron las reglas de negocio directamente a estos objetos usando el decorador **@field_validator**. De esta manera, si un cliente de la API envía un rechazo sin justificación, el DTO falla inmediatamente en la capa de red (con un error HTTP 422 Automático), impidiendo que los datos inválidos siquiera alcancen la capa de servicios o el repositorio.

3.4.4. Código Refactorizado (Después)

El código fue refactorizado centralizando la responsabilidad de validación (SRP) y reduciendo las firmas de los repositorios a recibir únicamente el ID y el objeto validado (payload):

```
# app/domain/schemas.py (Código Refactorizado - DTOs de
Entrada)
from pydantic import BaseModel, field_validator,
ValidationInfo

class DictamenInput(BaseModel):
    """
    DTO de Entrada (Aprobador): Payload para registrar la
    decisión final.
    Implementa ISP (Segregación de Interfaces) al solicitar
    solo campos pertinentes.
    """
    decision: str
    comentario: str = ""

    @field_validator('comentario')
    @classmethod
    def validar_comentario_obligatorio(cls, v: str, info:
ValidationInfo):
        """Aplica validación estricta de la Regla RN-03
        (Comentario obligatorio)."""
        decision = info.data.get('decision')
        if decision in ['OBSERVADO', 'RECHAZADO']:
            if not v or len(v.strip()) < 5:
                raise ValueError("RN-03: El comentario es
obligatorio y debe explicar el motivo (mín. 5 caracteres).")
            return v

class DerivacionInput(BaseModel):
    """
    DTO de Entrada (Secretario): Payload para el primer
    filtro técnico.
    Separa la responsabilidad del Secretario de la del
    Aprobador.
    """
    area_destino: str
    checklist_valido: bool
    comentario: str = ""

    @field_validator('comentario')
    @classmethod
    def validar_comentario_secretario(cls, v: str, info:
ValidationInfo):
        """Aplica validación estricta de la Regla RN-03 en
        caso de requisitos incompletos."""
```

```

        checklist = info.data.get('checklist_valido')
        if checklist is False:
            if not v or len(v.strip()) < 5:
                raise ValueError("RN-03: El comentario es
obligatorio al observar por falta de requisitos.")
            return v

```

3.5. Caso 5: *Message Chains* (Violación de la Ley de Demeter)

3.5.1. *Diagnóstico e Impacto*

Durante la revisión de los controladores de la API, se identificó que el código de presentación accedía a los datos de las solicitudes encadenando múltiples llamadas a propiedades y relaciones del ORM (por ejemplo, navegando desde la solicitud hacia su estado, y luego extrayendo el nombre del estado: `solicitud.estado_actual.tipoEstado`).

Diagnóstico: Este anti-patrón se conoce como Message Chains (Cadenas de Mensajes) y ocurre cuando un cliente solicita a un objeto otro objeto, y a este último otro, sucesivamente.

Impacto: Violación directa de la Ley de Demeter. El controlador de la API estaba íntimamente acoplado a la estructura jerárquica interna de la base de datos (SQLAlchemy). Si el modelo de datos cambiaba (por ejemplo, si la tabla estados se normalizaba o cambiaba de nombre), el controlador colapsaría. La capa de presentación "sabía demasiado" sobre cómo estaban contruidos los datos por debajo.

3.5.2. *Código Original (Antes)*

```

# app/api/approval_controller.py (Versión Inicial con Code
Smell)
@router.get("/approvals/{id}/detail")
def ver_detalle_solicitud(id: int, db: Session =
Depends(get_db)):
    solicitud =
db.query(Solicitud).filter(Solicitud.idSolicitud ==
id).first()

    # SMELL: Message Chain (Navegación profunda:
a.getB().getC())
    # El controlador conoce la existencia de la relación
'estado_actual' y su atributo 'tipoEstado'
    estado_texto = solicitud.estado_actual.tipoEstado
    fecha_historial = solicitud.historial_decisiones[-
1].fecha

    return {
        "id": solicitud.idSolicitud,

```

```
"estado": estado_texto,  
"ultima_modificacion": fecha_historial  
}
```

3.5.3. Técnica de Refactorización Aplicada

Para resolver este acoplamiento estructural, se aplicó la técnica de refactorización Hide Delegate (Ocultar Delegado).

Se determinó que el controlador no debía ser responsable de "navegar" por el ORM. En su lugar, esta responsabilidad se trasladó a las capas inferiores (la Fachada y los DTOs de Pydantic). La capa de Aplicación/Servicios ahora se encarga de resolver estas relaciones (aplanamiento de datos) y le entrega al controlador un objeto plano de tipo SolicitudDTO. De este modo, se oculta el delegado (el modelo ORM) y el controlador interactúa únicamente con su dependencia directa (el DTO).

3.5.4. Código Refactorizado (Después)

El controlador ya no navega por las relaciones. Simplemente invoca a la Fachada, la cual mapea las entidades complejas en estructuras de un solo nivel:

```
# app/services/bandeja_facade.py (El Delegado Oculto)  
class BandejaAprobacionFacade:  
    def obtener_bandeja_ordenada(self) -> list[SolicitudDTO]:  
        solicitudes_db =  
solicitud_repository.listar_solicitudes_por_aprobar(self.db)  
        dto_list = []  
        for sol in solicitudes_db:  
            # La Fachada resuelve la Message Chain  
internamente y aplanar el dato  
            dto = SolicitudDTO(  
                id=sol.idSolicitud,  
                tipo_tramite=sol.tipoSolicitud,  
                estado=sol.estado_actual.tipoEstado, #  
Aplanamiento  
                prioridad=sol.prioridad  
            )  
            dto_list.append(dto)  
        return dto_list  
  
# app/api/approval_controller.py (Controlador Refactorizado)  
@router.get("/approvals/pending",  
response_model=List[SolicitudDTO])  
def listar_pendientes(db: Session = Depends(get_db)):  
    fachada = BandejaAprobacionFacade(db)  
  
    # El controlador recibe una lista de DTOs planos.
```



```
# Ya no sabe, ni le importa, si el 'estado' viene de una
tabla relacionada o no.
return fachada.obtener_bandeja_ordenada()
```

4. Verificación y Métricas de Calidad

La refactorización de código no puede considerarse exitosa si altera el comportamiento externo del sistema o introduce regresiones. Por ello, todos los cambios estructurales aplicados al Módulo de Aprobaciones de Campus360 fueron respaldados y validados mediante un enfoque sistemático de aseguramiento de calidad.

4.1. Validación de la refactorización (Pruebas Unitarias)

Para garantizar que la reestructuración del código (extracción de métodos, creación de fachadas y aplicación de patrones) fuera segura, se ejecutó la suite de pruebas automatizadas utilizando el framework **pytest**.

El beneficio inmediato de haber aplicado el Principio de Inversión de Dependencias (DIP) y de haber eliminado el acoplamiento directo con MySQL en los controladores, fue la capacidad de inyectar dependencias falsas (Mocks). Se utilizaron objetos simulados (**unittest.mock**) para el **solicitud_repository**, permitiendo probar la lógica de negocio de la **BandejaAprobacionFacade** y los algoritmos de las estrategias de SLA de manera completamente aislada.

Tras finalizar los 5 casos de refactorización documentados, la suite de pruebas unitarias se ejecutó con un 100% de éxito (pasando en verde). Esto certifica empíricamente que se erradicaron los Code Smells sin romper los flujos de negocio existentes ni alterar los contratos de la API REST.

4.2. Mejoras medibles en la arquitectura

La intervención técnica generó un impacto positivo y cuantificable en la salud del código fuente. Para evidenciar objetivamente la calidad alcanzada, se extrajeron las siguientes métricas comparativas que muestran el "Antes y Después" de la refactorización:

Métrica de Calidad de Código	Estado Original (Antes)	Estado Refactorizado (Después)	Impacto Logrado
Complejidad Ciclomática (Cálculo SLA)	Múltiples ramas (if/elif/else anidados)	1 delegación polimórfica	Reducción drástica de complejidad ($O(1)$ en el orquestador).
Volumen de Código (Controlador API)	~ 125 líneas (<i>Fat Controller</i>)	~ 35 líneas (<i>Thin Controller</i>)	Mejora exponencial de legibilidad y

			cumplimiento SRP.
Duplicación de Código (Bloques JSON)	4 bloques iterativos redundantes	0 bloques (Centralizado en DTO)	100% de cumplimiento del principio DRY.
Acoplamiento (Capa API a Base de Datos)	Fuerte (<i>Message Chains</i> en ORM)	Nulo (Desacoplado vía Facade)	Cumplimiento estricto de la Ley de Demeter.

Estas métricas cuantitativas se traducen en los siguientes beneficios estructurales:

- Reducción de la Complejidad Ciclomática: Al reemplazar las extensas cadenas de condicionales anidados (if/elif/else) por el patrón Strategy polimórfico, la complejidad algorítmica en la toma de decisiones de SLA se redujo drásticamente. El código pasó de evaluar múltiples ramas de ejecución a un flujo lineal y predecible.
- Eliminación de Duplicidad (Cumplimiento DRY): La adopción de Data Transfer Objects (DTOs) validados con Pydantic erradicó la creación manual de diccionarios JSON iterativos. Esto redujo significativamente la cantidad de líneas de código en los controladores y centralizó el mantenimiento de los contratos de salida en un único archivo de esquemas ([schemas.py](#)).
- Aumento de la Cohesión y Legibilidad: Al delegar responsabilidades a la capa de Servicios y establecer fronteras claras (DTOs para entrada/salida y ORM exclusivo para persistencia), los componentes ahora tienen una única razón para cambiar (SRP). El código es ahora auto-documentado, expresivo y está preparado para escalar funcionalmente en las próximas fases del proyecto integrador.